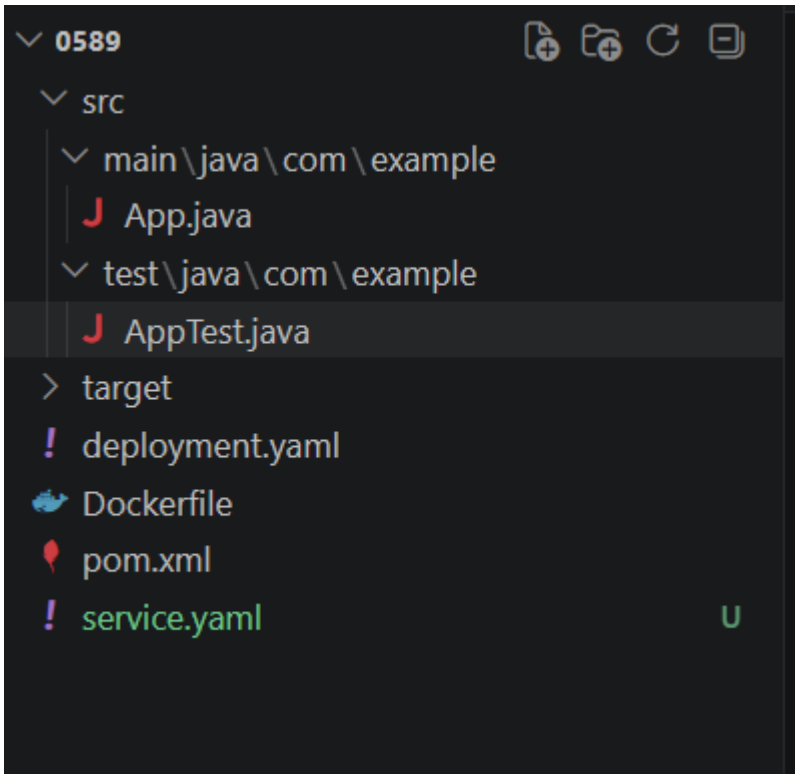
Step1:



Paste this in inside App.java:

```
package com.example;


public class App {


    // Method to test
    public int add(int a, int b) {
        return a + b;
    }
}
```

Paste this inside AppTest.java:

```
package com.example;

import org.junit.Test;
import static org.junit.Assert.assertEquals;

public class AppTest {

    @Test
    public void testAdd() {
        App app = new App();

        int result = app.add(2, 3);

        assertEquals(5, result);
    }
}
```

Deployment.yaml:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app-deployment        # 🔹 Change this
  labels:
    app: my-app
```

```yaml
spec:
  replicas: 2                    # 🔹 Number of pods
  selector:
    matchLabels:
      app: my-app

  template:
    metadata:
      labels:
        app: my-app

    spec:
      containers:
      - name: my-app-container     # 🔹 Container name
        image: nginx:latest        # 🔹 Change to your image
        ports:
        - containerPort: 80        # 🔹 App port

        # 🔹 Optional: Environment variables
        env:
        - name: ENV
          value: "production"

        # 🔹 Optional: Resource limits
        resources:
          requests:
```

```
          memory: "64Mi"
          cpu: "250m"
        limits:
          memory: "128Mi"
          cpu: "500m"

      # 🔹 Optional: Liveness probe
      livenessProbe:
        httpGet:
          path: /
          port: 80
        initialDelaySeconds: 15
        periodSeconds: 20

      # 🔹 Optional: Readiness probe
      readinessProbe:
        httpGet:
          path: /
          port: 80
        initialDelaySeconds: 5
        periodSeconds: 10
```

service.yaml:

```
apiVersion: v1
kind: Service
metadata:
  name: my-app-service       # 🔹 Change this
  labels:
```

```
    app: my-app

spec:
  type: ClusterIP          # 🔹 Default (internal access)
  selector:
    app: my-app            # 🔹 Must match deployment labels

  ports:
    - protocol: TCP
      port: 80             # 🔹 Service port
      targetPort: 80       # 🔹 Container port
```

pom.xml:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
        http://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>com.example</groupId>
    <artifactId>simple-maven-app</artifactId>
    <version>1.0-SNAPSHOT</version>

    <dependencies>
        <!-- JUnit 5 -->
        <dependency>
            <groupId>org.junit.jupiter</groupId>
            <artifactId>junit-jupiter</artifactId>
            <version>5.10.2</version>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <!-- Required for JUnit 5 -->
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
```

```
            <artifactId>maven-surefire-plugin</artifactId>
            <version>3.2.5</version>
        </plugin>
    </plugins>
  </build>

</project>
```

Dockerfile:

```
FROM eclipse-temurin:17-jdk
WORKDIR /app
COPY target/simple-java-app-1.0.jar app.jar
CMD ["java", "-jar", "app.jar"]
```

Step2:

mvn clean test

output:

build success

step3:

```
git init
git add .
git commit -m "Initial Maven project with JUnit test"
git branch -M main
git remote add origin https://github.com/avinasharavindu9-coder/avinashFat.git
git push -u origin main
```

jenkins:

settings->credentials->global->username with password

id: dockerhub-creds

Step4:

```
pipeline {
    agent any

    environment {
        DOCKER_IMAGE = "23mis0654/java-app:latest"
    }

    stages {

        stage('Clone Repository') {
            steps {
                git branch: 'main',
                    url: 'https://github.com/avinasharavindu9-coder/avinashFat.git'
            }
        }

        stage('Build with Maven') {
            steps {
                bat 'mvn clean package -DskipTests'
            }
        }

        stage('Build Docker Image') {
```

```
    steps {
        bat "docker build -t %DOCKER_IMAGE% ."
    }
}

stage('Push to DockerHub') {
    steps {
        withCredentials([
            usernamePassword(
                credentialsId: 'dockerhub-creds',
                usernameVariable: 'DOCKER_USER',
                passwordVariable: 'DOCKER_PASS'
            )
        ]) {
            bat """
            echo %DOCKER_PASS% | docker login -u %DOCKER_USER% --password-stdin
            docker push %DOCKER_IMAGE%
            """
        }
    }
}

stage('Deploy to Kubernetes') {
    steps {
        withCredentials([
            file(credentialsId: 'kubeconfig-secret', variable: 'KUBECONFIG')
        ]) {
            bat """
```

```
                kubectl apply -f deployment.yaml
                kubectl apply -f service.yaml
                """
            }
        }
    }
}

    post {
        success {
            echo 'CI/CD Pipeline executed successfully!'
        }
        failure {
            echo 'Pipeline failed. Please check logs.'
        }
    }
}
```

Step5:

Settings->credentials->global->secret file
put path
id: kuberconfig-helo

Now go tap build now
take screenshots

Step 6:

kubectl get svc